南開大學

信息安全综合实验课程实验报告

实验六：简单入侵检测模型的设计与实现

学　　院　密码与网络空间安全学院

专　　业　信息安全、法学双学位班

学　　号　2313815

姓　　名　段俊宇

班　　级　信息安全、法学双学位班

一、实验目的

入侵检测系统（Intrusion Detection System，简称 IDS）是一种对网络传输进行实时监视，并在发现可疑传输时发出警报或者采取主动防御措施的网络安全设备。作为 IDS 的核心技术，入侵检测通过收集和分析网络行为、安全日志、审计数据以及计算机系统中若干关键点的信息来检查网络与系统中是否存在违反安全策略的行为和被攻击的迹象。根据不同的实现方法，入侵检测分为特征检测与异常检测。特征检测假设入侵行为之间存在相似的特征。它通过对已有入侵行为进行收集和分析，提取出共同的特征，然后利用这些特征预测新的入侵行为。

本实验编程训练的目的如下：

① 深入理解特征检测的基本思想，以及其在入侵检测系统中的应用。

② 掌握 K-Means 算法的计算过程。

③ 掌握数据挖掘的基本概念与方法。

本实验编程训练的要求如下：

① 使用 K-Means 算法对 KDD Cup 1999 数据集进行聚类分析，建立简单的入侵检测模型。

② 利用入侵检测模型对测试数据进行预测。

二、实验原理

1. KDD Cup 1999 数据集

KDD Cup 1999 数据集是第 3 届知识发现与数据挖掘竞赛所使用的数据集。该数据集来源于一个模拟的美国空军军事网络环境下的局域网流量数据，包含了多种网络攻击与入侵行为。KDD Cup 1999 通过记录上述局域网连续 9 周的原始 TCP dump 数据流量，生成大约 700 万条连接记录。每一条连接

记录反映了某一时刻从源 IP 地址到目的 IP 地址若干数据包的传输情况。整个数据集分为训练数据集和测试数据集 2 部分。前者用于提取数据特征，生成数据挖掘模型，后者用于验证模型的效率与正确性。

KDD Cup 1999 对每一条连接记录都进行了分类（label）。所有记录分为正常（normal）与攻击（attack）大两类。其中攻击（attack）行为又分为以下 4 类：

(1) DOS：拒绝服务，例如，SYN 洪泛攻击。

(2) R2L：来自于远程主机的非法入侵，例如，猜测密码。

(3) U2R：非法获得本地主机的超级用户权限，例如，各种“缓冲区溢出”攻击。

(4) Probing：监视与探测，例如，端口扫描。

除类别（label）以外，KDD Cup 1999 数据集中的每一条连接记录还包含了 41 项属性。

本实验共用到其中 18 项属性，如表 10-1 所示。其中前 3 个属性为符号类型，其它属性为连续类型。

表 10-1　KDD Cup 1999 数据记录的 18 项属性

名称	类型	说明
protocol_type	symbolic	协议类型
Service	symbolic	服务类型
Flag	symbolic	状态标志
src_bytes	continuous	源到目的字节数
dst_bytes	continuous	目的到源字节数
num_failed_logins	continuous	登录失败次数
num_root	continuous	root 用户权限存取次数
Count	continuous	2 秒内连接相同主机数目
srv_count	continuous	2 秒内连接相同端口数目
serror_rate	continuous	"REJ"错误的连接数比率
same_srv_rate	continuous	连接到相同端口数比率
diff_srv_rate	continuous	连接到不同端口数比率
dst_host_srv_count	continuous	相同目的地相同端口连接数
dst_host_same_srv_rat	continuous	相同目的地相同端口连接数比率
dst_host_diff_srv_rate	continuous	相同目的地不同端口连接数比率
dst_host_same_src_port_rate	continuous	相同目的地相同源端口连接比率
dst_host_srv_diff_host_rate	continuous	不同主机连接相同端口比率
dst_host_srv_serror_rate	continuous	连接当前主机有 S0 错误的比率

http://archive.ics.uci.edu/ml/databases/kddcup99/kddcup99.html 可以下载 KDD Cup 1999 数据集。值得注意的是，KDD Cup 1999 作为一个

完备的数据集，包含了多个文件。

本实验编程将使用文件kddcup.data_10_percent.gz与corrected.gz分别作为入侵检测模型的训练数据集与测试数据集。

2. K-Means 算法

1. 聚类方法简介

K-Means 算法是一种聚类分析方法。目前在聚类分析领域，根据具体应用的不同，聚类算法分为以下 5 类：

（1）划分方法

（2）层次方法

（3）基于密度的方法

（4）基于网格的方法

（5）基于模型的方法。

本实验使用的 K-Means 算法就是划分方法中的一种。所谓划分方法，就是在一个由 n 个对象或元组组成的数据库中构建数据的 k 个划分（k<n），每个划分表示一个聚类。也就是说，本方法将数据划分为 k 个聚类，并且同时满足以下 2 个要求：

（1）每个聚类至少包含一个对象。

（2）每个对象必须属于且只属于一个聚类。

一般来说，判断划分结果优劣的基本原则是：同一个聚类中的对象之间尽可能“接近”，不同聚类中的对象之间尽可能“远离”。通常，聚类算法会选择一个划分标准（称为相似度函数）来判定对象之间的相似度，比如距离等。下面首先介绍聚类分析中的数据类型以及相似度的计算方法，然后再讨论 K-Means 算法的设计与实现。

2. 聚类分析中的相似度计算方法

聚类分析中的数据类型包括数值属性、二值属性、符号属性、顺序属性和比例属性。因为 K-Means 算法只适用于聚类均值有意义的情况，所以本实验只介绍数值属性的相似度计算方法。数值属性所描述对象之间的相似度可以通过计算两个数据对象之间距离来确定，比如欧氏距离、Manhattan 距离以及Minkowski距离等。最常用的距离计算公式就是欧氏距离，定义如下：

$$d(i,j)=\sqrt{\left(\left|x_{i1}-x_{j1}\right|^2+\left|x_{i2}-x_{j2}\right|^2+\cdots+\left|x_{\mathrm{ip}}-x_{\mathrm{jp}}\right|^2\right)}$$

其中对象$i=\left(x_{i1},x_{i2},\ldots,x_{ip}\right)$；对象$j=\left(x_{j1},x_{j2},\ldots,x_{jp}\right)$。

另一个常用的距离计算方法就是计算 Manhattan 距离，公式定义如下：

$$d(i,j)=\left|x_{i1}-x_{j1}\right|+\left|x_{i2}-x_{j2}\right|+\cdots+\left|x_{\mathrm{ip}}-x_{\mathrm{jp}}\right|$$

Minkowski 距离是欧式距离和 Manhattan 距离的一个推广，公式定义如下：

$$d(i,j)=\left(\left|x_{i1}-x_{j1}\right|^q+\left|x_{i2}-x_{j2}\right|^q+\cdots+\left|x_{\mathrm{ip}}-x_{\mathrm{jp}}\right|^q\right)^{\frac{1}{q}}$$

其中 q 为一个正整数；当 q=1 时，它代表 Manhattan 距离计算公式；而当 q=2 时，它代表欧氏距离计算公式。

若每个变量被赋予一个权值 w，表示其所代表属性的重要性，那么带权值的欧氏距离计算公式如下：

$$d(i,j)=\sqrt{\left(w_1\left|x_{i1}-x_{j1}\right|^2+w_2\left|x_{i2}-x_{j2}\right|^2+\cdots+w_p\left|x_{\mathrm{ip}}-x_{\mathrm{jp}}\right|^2\right)}$$

同样，Manhattan 距离和 Minkowski 距离也可以引入权值进行计算。

3. K-Means 算法设计

(1) K-Means 算法基本思想

K-Means 算法首先随机选取 k 个对象作为初始聚类中心；然后计算各个数据对象到每个聚类中心的距离，把数据对象划分到离它最近的聚类中心所在的类中；对调整后的新类重新计算聚类中心，如果相邻两次计算的聚类中心没有任何变化，说明数据对象的划分结束。本算法的特点是在每次迭代计算中都要考察各个对象的分类是否正确，如果不正确，就需要重新进行分类。在全部数据调整完毕后，再重新计算聚类中心，进入下一次迭代。如果在一次迭代后所有的数据对象都被正确归类，那么聚类中心将不会改变。这也标志着算法已经达到收敛，可以终止聚类过程。

(2) K-Means 算法流程

K-Means 算法的流程用数学语言描述如下。

a. 给定大小为 n 的数据集，令聚类次数 I=1，选取 k 个数据对象作为初始聚类中心$Z_j(I),j=1,2,3,\ldots,k$。k个聚类用$Cluter(j)$表示，其中$j=$

$1,2,3,\dots,k$。

b. 计算每个数据对象与各聚类中心的距离$D\left(x_i, Z_j(I)\right), i = 1,2,3,\dots,n, j = 1,2,3,\dots,k$，如果满足$D\left(x_i, Z_m(I)\right) = min\{D\left(x_i, Z_j(I)\right), j = 1,2,3,\dots,k\}$，那么$x_i \in Cluter(m)$。

c. 按照下式重新计算每个聚类的聚类中心。其中n_j为$Cluter(j)$中数据对象的数目，$x_i^{(j)}$表示 $Cluter(j)$中第 i 个对象的值。

$$Z_j(I+1) = \frac{1}{n_j}\sum_{i=1}^{n_j} x_i^{(j)}, (j = 1,2,\dots,k)$$

d. 若 $Z_j(I+1) \neq Z_j(I)$，则 $I = I + 1$，返回第 2 步；否则，算法结束。

3. K-Means 算法的缺陷与扩展

1. K-Means 算法的缺陷

虽然 K-Means 算法复杂度低，易于实现，但是在处理大规模数据时，聚类效果往往欠佳。尤其是在对初始聚类中心数目 k 进行设置时，选取不同的 k 值，聚类结果也大不相同。KDD Cup 1999 数据集除了包含大量的正常数据外，还包含了多种攻击数据。虽然这些攻击数据可以划分为 4 大类，但是在使用 K-Means 算法进行聚类时，仅仅将初始聚类中心数目设置成 5（包括正常数据类型的情况）并不能够获得最佳的聚类效果。因为在 KDD Cup 1999 数据集中，正常数据和攻击数据是多种多样的。以 DOS 攻击为例，它包含了 smurf、neptune、back、teardrop、pod 以及 land 共 6 种攻击方式。不同种类的攻击有着不同的数值特性，即使属于同一类攻击方式，在相似度方面也存在很大差异。有些攻击数据在相似度上与正常数据十分接近，却与同类型的攻击数据相去甚远。因此，在使用 K-Means 算法对 KDD Cup 1999 进行聚类分析时，无论 k 值如何选取，仅仅对数据集进行一次聚类是远远不够的。

2. K-Means 算法扩

为了获得更加精确的聚类结果，需要利用 K-Means 算法对 KDD Cup 1999 数据集进行多次聚类。

对一次聚类而言，它的结果包含了多个划分，每一个划分表示一个子类（SubCluster）。每一个子类中可能只包含了一种类别的数据，也可能包含了多种类别的数据。对于前一种情况而言，说明该子类已经从其它数据中划分出来，代表了唯一一种类别；而对于后一种情况而言，子类中包含着干扰数据，不能用一种类别来对子类进行标记。在一般情况下，一次聚类很难划分出若干个完全干净（即不含任何干扰数据）的子类。子类中往往不只含有一种类别的数据。因此判断一个子类是否可以代表一种类别，是否需要再进行一次聚类，需要引入聚类精度的概念。

聚类精度（Cluster Precision）表示一个子类中干扰数据占主类别数据的比例。它的计算过程分为以下 5 步。

(1) 统计子类中不同类别（Label）数据的数目，用 LabelNum[1]、LabelNum [2]……表示。

(2) 在所有类别中找出数据数目最大的一项，将该值记为 MaxLabelNum，并将对应的类别设置为本子类的主类别（MasterLabel）。

(3) 统计所有非主类别的数据的数目，记为 OtherLabelNum。

(4) 计算子类的聚类精度 Cluster Precition = OtherLabelNum/MaxLabelNum。

(5) 判断聚类精度是否符合要求（即是否低于某一精度标准，比如 0.1），若不是，则使用 K-Means 算法对该子类继续进行聚类；否则，用主类别标记该子类。

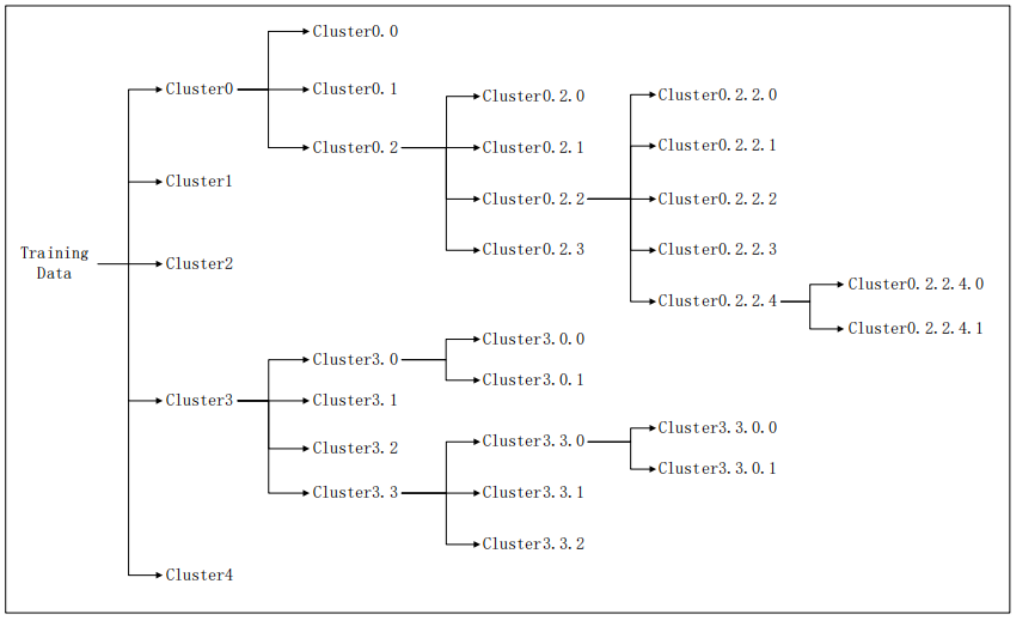

图 1 聚类树

图 1 形象地描述了利用 K-Means 算法进行多次聚类的过程。根节点表示训练数据集。通过 K-Means 算法对其进行一次聚类后，划分出 5 个子类，Cluster0、…、Cluster4。通过计算聚类精度，得出 Cluster1、Cluster2 和 Cluster4 符合精度要求，不再进行任何操作；而 Cluster1 和 Cluster4 不符合要求，需要采用 K-Means 算法继续进行聚类。通过重复上述操作，不符合聚类精度要求的子类不断地被进一步划分，生成了一棵聚类树（Cluster Tree）。树中的每一个叶子节点都代表了一个符合聚类精度要求的子类。聚类树的深度则代表了采用 K-Means 算法进行聚类的次数。

此外，采用 K-Means 算法进行多次聚类是一个自动的过程。对于每次聚类而言，初始聚类中心数目 k 的设定也应该是自动的，不需要用户进行手动输入。因此，本实验规定在对训练数据集进行首次 K-Means 聚类时，将 k 值设置为 5。这样既包含了正常类别，也包含了 4 种攻击类别。在对子类进行 K-Means 聚类时，k 值设为该子类中所包含的不同类别数目。采用这种取值方案，是希望在下一次聚类后将子类划分为不同类别的 k（k≤5）个聚类。

三、实验环境

Windows Subsystem for Linux 操作系统，g++、gunzip 工具，kddcup.data_10_percent 和 corrected 数据集。

四、实验过程

1. 环境配置

首先通过 https://www.kaggle.com/datasets/galaxyh/kdd-cup-1999-data?resource=download 下载数据集，并在 WSL 中使用 gunzip 解压缩，然后需要使用 sudo apt install g++ make -y 安装编译工具。

2. 编写代码

如下所示是补充的 Kmeans.cpp 的 main 函数内容，我将使用注释的形式进行解释。

```
int main()
{
    int Kvalue;                                  //K 值
    int iRightRcdNum;                            //记录分类正确的记录数
    int TestRcdNum;                              //参加检测的记录总数
    string strTrueLabel,strPreLabel;              //真实标签与预测标签

    strMyRecord* pRecord;                        //遍历记录的指针
    list <strMyRecord*>* pTestRcdList;            //测试数据链表
    list<strMyRecord*>::iterator TestListIter;     //记录链表的迭代器

    ClusterNode* pClusterNode;                   //聚类节点指针
    ClusterTree* pClusterTree;                   //聚类树


    /*****************************************************************************
    *********** PART1 利用 K-Means 算法，对训练数据集进行迭代聚类，并创建聚类树
    ************
    ******************************************************************************
    ******/

    //创建聚类树
    pClusterTree = new ClusterTree();

```

```
22.     //创建 KMeans 聚类对象
23.     //根节点层次为 1，数据维数为 19（18 个属性+1 个标签）
24.     CKMeans* pRootKMeans = new CKMeans(pClusterTree, ++KmeansID, 1, 19);
25.
26.     //读取训练集中的记录
27.     cout<<"========== Step 1: Reading training data ==========="<<endl;
28.     outfile<<"========== Step 1: Reading training data ==========="<<endl;
29.     if(!pRootKMeans->ReadTrainingRecords())
30.     {
31.             cout<<"Failed to read training data!"<<endl;
32.             return -1;
33.     }
34.
35.     //用户输入 K 值或直接将 K 值设置为 5
36.     Kvalue = 5;
37.     cout<<"========== Step 2: Setting K value = "<<Kvalue<<" ==========="<<endl;
38.     outfile<<"========== Step 2: Setting K value = "<<Kvalue<<" ==========="<<endl;
39.
40.     //运行 Kmeans 算法
41.     cout<<"========== Step 3: Running K-Means algorithm ==========="<<endl;
42.     outfile<<"========== Step 3: Running K-Means algorithm ==========="<<endl;
43.     pRootKMeans->RunKMeans(Kvalue);
44.
45.     //打印聚类树
46.     cout<<"========== Step 4: Printing cluster tree ==========="<<endl;
47.     outfile<<"========== Step 4: Printing cluster tree ==========="<<endl;
48.     pClusterTree->Print();
49.
50.     //将聚类树打印至日志文件中
51.     pClusterTree->PrintLog();
52.     cout<<"Cluster tree has been saved to Log.txt"<<endl;
53.
54.     /*****************************************************************************
**********
55.     ******* PART2 利用聚类树 对测试数据集中的数据进行类型预测，并计算出准确率和混淆矩阵
******
56.     *****************************************************************************
*********/
57.
58.     //读取测试文件
59.     cout<<"========== Step 5: Reading test data ==========="<<endl;
60.     outfile<<"========== Step 5: Reading test data ==========="<<endl;
```

```
61.     pTestRcdList = new list<strMyRecord*>();
62.     if(!ReadTestFile(pTestRcdList))
63.     {
64.             cout<<"Failed to read test data!"<<endl;
65.             return -1;
66.     }
67.
68.     //创建一个混淆矩阵
69.     ConfuseMatrix cfMatrix;
70.
71.     //预测正确的记录数设置为 0
72.     iRightRcdNum = 0;
73.
74.     //将检测记录数设置为 0
75.     TestRcdNum = 0;
76.
77.     //遍历测试数据链表，为每一个数据找到聚类中心
78.     cout<<"========== Step 6: Predicting test data ==========="<<endl;
79.     outfile<<"========== Step 6: Predicting test data ==========="<<endl;
80.     Rstfile<<"========== Prediction Results ==========="<<endl;
81.
82.     for(TestListIter = pTestRcdList->begin(); TestListIter != pTestRcdList->end();
TestListIter++)
83.     {
84.             pRecord = (*TestListIter);
85.             TestRcdNum++;
86.
87.             //找到与当前记录距离最近的聚类节点
88.             pClusterNode = pClusterTree->FindNearestCluster(pRecord);
89.
90.             //获取预测的类别标签
91.             int iPreLabel = pClusterNode->GetClusterNodeLabel();
92.             int iTrueLabel = pRecord->iLabel;
93.
94.             //统计分类正确的记录数
95.             if(iPreLabel == iTrueLabel)
96.             {
97.                     iRightRcdNum++;
98.             }
99.
100.            //更新混淆矩阵
101.            cfMatrix.UpdateValue(iTrueLabel, iPreLabel);
102.
103.            //将分类记录写到 result.txt 文件中
```

```
                strTrueLabel = LabelInttoStr(iTrueLabel);
                strPreLabel = LabelInttoStr(iPreLabel);

                Rstfile<<"Record "<<TestRcdNum<<": True = "<<strTrueLabel
                      <<", Predict = "<<strPreLabel;
                if(iPreLabel == iTrueLabel)
                {
                          Rstfile<<" [OK]"<<endl;
                }
                else
                {
                          Rstfile<<" [WRONG]"<<endl;
                }

                //每 10000 条记录打印一次进度
                if(TestRcdNum % 10000 == 0)
                {
                          cout<<setiosflags(ios::left)<<setw(8)<<TestRcdNum<<" records have been predicted ..."<<endl;
                }
        }

        //计算正确分类率
        float fAccuracy = (float)iRightRcdNum / TestRcdNum * 100;

        //打印屏显
        cout<<"========== Step 7: Results Summary ==========="<<endl;
        cout<<"Total test records: "<<TestRcdNum<<endl;
        cout<<"Correctly predicted: "<<iRightRcdNum<<endl;
        cout<<"Accuracy: "<<fAccuracy<<"%"<<endl;

        //写入日志文件
        outfile<<"========== Step 7: Results Summary ==========="<<endl;
        outfile<<"Total test records: "<<TestRcdNum<<endl;
        outfile<<"Correctly predicted: "<<iRightRcdNum<<endl;
        outfile<<"Accuracy: "<<fAccuracy<<"%"<<endl;

        //打印混淆矩阵
        cout<<"========== Confusion Matrix ==========="<<endl;
        cfMatrix.PrintMatrix();

        //将混淆矩阵输出到结果文件中
        cfMatrix.PrintMatrixToLog();

```

```
    cout<<"========== Program Finished ==========="<<endl;
    cout<<"Results have been saved to Result.txt and Log.txt"<<endl;

    //清理内存
    delete pRootKMeans;
    delete pClusterTree;
    delete pTestRcdList;

    return 0;
}
```

此外，还需要在 Commom.h 中添加 cstring 和 cstdlib 头文件，否则会编译报错。接下来使用下面的命令编译代码：

```
# 编译 DataPretreat 程序
g++ -o DataPretreat ./ DataPretreat/DataPreTreat.cpp -std=c++11

# 编译 Kmeans 程序
g++ -o Kmeans ./KMeans/Kmeans.cpp ./KMeans/ClusterTree.cpp -std=c++11 -lm
```

```
➜ project g++ -o Kmeans ./KMeans/Kmeans.cpp ./KMeans/ClusterTree.cpp -std=c++11 -lm
In file included from ./KMeans/Kmeans.cpp:1:
./KMeans/Kmeans.h: In member function 'bool CKMeans::IsClusterOK(int)':
./KMeans/Kmeans.h:824:1: warning: control reaches end of non-void function [-Wreturn-type]
  824 | }
      | ^
```

然后需要预处理训练数据和测试数据，命令如下：

```
# 第一步：预处理训练数据
./DataPretreat kddcup.data_10_percent

# 第二步：预处理测试数据
./DataPretreat corrected
```

```
Start writing records into kddcup.data_10_percent_datatreat ...
---------- 10000   lines have written ----------
---------- 20000   lines have written ----------
---------- 30000   lines have written ----------
---------- 40000   lines have written ----------
---------- 50000   lines have written ----------
---------- 60000   lines have written ----------
---------- 70000   lines have written ----------
---------- 80000   lines have written ----------
---------- 90000   lines have written ----------
---------- 100000  lines have written ----------
---------- 110000  lines have written ----------
---------- 120000  lines have written ----------
---------- 130000  lines have written ----------
---------- 140000  lines have written ----------
---------- 150000  lines have written ----------
---------- 160000  lines have written ----------
---------- 170000  lines have written ----------
---------- 180000  lines have written ----------
---------- 190000  lines have written ----------
---------- 200000  lines have written ----------
---------- 210000  lines have written ----------
---------- 220000  lines have written ----------
---------- 230000  lines have written ----------
---------- 240000  lines have written ----------
---------- 250000  lines have written ----------
---------- 260000  lines have written ----------
---------- 270000  lines have written ----------
---------- 280000  lines have written ----------
---------- 290000  lines have written ----------
---------- 300000  lines have written ----------
---------- 310000  lines have written ----------
---------- 320000  lines have written ----------
---------- 330000  lines have written ----------
---------- 340000  lines have written ----------
---------- 350000  lines have written ----------
---------- 360000  lines have written ----------
---------- 370000  lines have written ----------
---------- 380000  lines have written ----------
---------- 390000  lines have written ----------
---------- 400000  lines have written ----------
---------- 410000  lines have written ----------
---------- 420000  lines have written ----------
---------- 430000  lines have written ----------
---------- 440000  lines have written ----------
---------- 450000  lines have written ----------
---------- 460000  lines have written ----------
---------- 470000  lines have written ----------
---------- 480000  lines have written ----------
---------- 490000  lines have written ----------
All Records have written !
```

```
---------- 160000   lines have read ----------
---------- 170000   lines have read ----------
---------- 180000   lines have read ----------
---------- 190000   lines have read ----------
---------- 200000   lines have read ----------
---------- 210000   lines have read ----------
---------- 220000   lines have read ----------
---------- 230000   lines have read ----------
---------- 240000   lines have read ----------
---------- 250000   lines have read ----------
---------- 260000   lines have read ----------
---------- 270000   lines have read ----------
---------- 280000   lines have read ----------
---------- 290000   lines have read ----------
---------- 300000   lines have read ----------
---------- 310000   lines have read ----------
All Records have read ! Total 311029 records !
Start writing records into corrected_datatreat ...
---------- 10000   lines have written ----------
---------- 20000   lines have written ----------
---------- 30000   lines have written ----------
---------- 40000   lines have written ----------
---------- 50000   lines have written ----------
---------- 60000   lines have written ----------
---------- 70000   lines have written ----------
---------- 80000   lines have written ----------
---------- 90000   lines have written ----------
---------- 100000  lines have written ----------
---------- 110000  lines have written ----------
---------- 120000  lines have written ----------
---------- 130000  lines have written ----------
---------- 140000  lines have written ----------
---------- 150000  lines have written ----------
---------- 160000  lines have written ----------
---------- 170000  lines have written ----------
---------- 180000  lines have written ----------
---------- 190000  lines have written ----------
---------- 200000  lines have written ----------
---------- 210000  lines have written ----------
---------- 220000  lines have written ----------
---------- 230000  lines have written ----------
---------- 240000  lines have written ----------
---------- 250000  lines have written ----------
---------- 260000  lines have written ----------
---------- 270000  lines have written ----------
---------- 280000  lines have written ----------
---------- 290000  lines have written ----------
---------- 300000  lines have written ----------
---------- 310000  lines have written ----------
All Records have written !
```

接下来运行 Kmeans 进行训练和预测即可，运行结束的结果如下：

```
========== Step 6: Predicting test data ==========
10000     records have been predicted ...
20000     records have been predicted ...
30000     records have been predicted ...
40000     records have been predicted ...
50000     records have been predicted ...
60000     records have been predicted ...
70000     records have been predicted ...
80000     records have been predicted ...
90000     records have been predicted ...
100000    records have been predicted ...
110000    records have been predicted ...
120000    records have been predicted ...
130000    records have been predicted ...
140000    records have been predicted ...
150000    records have been predicted ...
160000    records have been predicted ...
170000    records have been predicted ...
180000    records have been predicted ...
190000    records have been predicted ...
200000    records have been predicted ...
210000    records have been predicted ...
220000    records have been predicted ...
230000    records have been predicted ...
240000    records have been predicted ...
250000    records have been predicted ...
260000    records have been predicted ...
270000    records have been predicted ...
280000    records have been predicted ...
290000    records have been predicted ...
300000    records have been predicted ...
310000    records have been predicted ...
========== Step 7: Results Summary ==========
Total test records: 311029
Correctly predicted: 299950
Accuracy: 96.438%
========== Confusion Matrix ==========
================================  Confusion Matrix  ================================
T/P  0      1      2      3      4
0    74775  3596   905    0      46
1    143    223136 19     0      0
2    218    120    2039   0      0
3    31     4      0      0      4
4    5442   535    16     0      0
========== Program Finished ==========
Results have been saved to Result.txt and Log.txt
```

聚类过程日志和聚类树信息保存在 log.txt 中，预测结果和混淆矩阵保存在 result.txt 中。从 log.txt 的内容我们可以看出总测试记录有 311029 条，其中成功预测的有 299950 条，准确率达到了 96.438%，说明程序没有问题，在已有代码基础上成功实现了对测试数据的预测！

五、实验结论及心得体会

本次实验我基于现有的代码，补全了 main 函数中的部分，最终实现了 K-Means 算法以及对测试数据的准确预测。在实验中，我理解了特征检测的基本思想以及 K-Means 聚类算法的基本流程，增强了我对数据挖掘技术在信息安全领域应用的理解。